

End-to-End MLOps Pipeline

Docker · GitHub Actions · Kaggle · W&B

MLOps | PGD AI Program | IIT Jodhpur

Total Marks: 100

Objective

Build a complete, production-style MLOps pipeline. Choose a reasonably small publicly available dataset and a compact pre-trained model from Hugging Face that suits your task. Clean and prepare the data, fine-tune the model, containerise inference with Docker, run multiple experiment versions on Kaggle, automate everything through GitHub Actions, and track all runs on Weights & Biases.

Group & Submission Instructions

Group Formation

- This is a group assignment, where the groups are auto-assigned on LMS
- All members must contribute commits to the repository — contributions are visible in the GitHub commit history and will be reviewed

Submission

Submit a single PDF report (4–5 pages) via the course portal. The report must contain —

- Name of the students along with their contributions
- Following live links (broken or private links will receive zero for that component):
 - GitHub Repository Link — public; must include workflow files
 - Kaggle Notebook Link(s) — one per experiment version, set to Public
 - Hugging Face Model Link — publicly accessible
 - Docker Image URL — publicly pullable
 - W&B Project Dashboard Link — project visibility set to Public
- Following specific details and screenshots:
 - Git repository setup — screenshot of collaborator roles and branch protection
 - Data cleaning decisions — what you did and why (missing values, normalisation strategy, class distribution)
 - Model selection rationale — why you chose this model (reference the HF model card)
 - Experiment comparison — table of Version 1 vs Version 2 metrics; which performed better and why
 - Screenshot or badge showing a successful GitHub Actions inference run
 - Docker design — describe your Dockerfile choices; paste the Actions log from a successful inference run
 - Challenges & learnings — what was hard, what you would do differently

⚠ Important: Plagiarism is strictly prohibited. Any work copied without attribution will result in zero marks for the entire assignment.

For Doubts & Queries

If you have any questions regarding the assignment, send an email to:

To	m25csa007@iitj.ac.in
CC	Hardik Jain
Subject	[MLOps Project] <Your Query>

✉ Note: Always include your group members' names and roll numbers in the email body. Emails without a proper subject line or group details may not receive a response.

Prerequisites

- Python (intermediate level)
- Basic Machine Learning concepts (training, evaluation, metrics)
- Git and GitHub
- Docker (free at [docker.com](https://www.docker.com))
- Hugging Face account (free at huggingface.co)
- Weights & Biases account (free at wandb.ai)
- Kaggle account (free at kaggle.com) — needed for GPU training

Task 1: Set Up the GitHub Repository [10 marks]

Create and configure the shared repository that will hold all project code.

- Create a new public GitHub repository with a README.md, .gitignore, and LICENSE [2 marks]
- Add a develop branch; protect main (require at least 1 pull-request review before merging) [3 marks]
- One team member is the **Admin** (repository owner); all others are added as **Collaborators** with Write access [3 marks]
- Include a screenshot of the Collaborators settings page in your report [2 marks]

 **Tip:** Set up the full folder structure and commit placeholder files early to avoid merge conflicts once everyone starts pushing code.

Task 2: Data Preparation & Normalisation [15 marks]

Select a publicly available dataset that suits your chosen task. This can be any modality — text, images, tabular data, or audio. Write a reusable Python script to prepare it for training.

- Inspect the raw data: size, structure, class distribution, and any quality issues [3 marks]
- Clean the data as appropriate for your modality — examples below [6 marks]

Modality	Typical cleaning / normalisation steps
Text	Lowercase, strip punctuation, remove duplicates, handle missing values
Images	Resize to a consistent shape, normalise pixel values (0–1 or ImageNet stats), remove corrupt files
Tabular	Impute or drop nulls, encode categoricals, scale numeric features
Audio	Resample to consistent rate, trim silence, convert to spectrograms if needed

- Encode labels and save an id2label.json mapping file [3 marks]
- Save the prepared dataset locally; commit only the mapping file (not large data files) [3 marks]

🔴 **Note:** Your dataset and task are your own choice — text classification, image classification, sentiment analysis, and similar well-scoped tasks all work well. Keep the dataset reasonably small (ideally under 50,000 samples) so training completes within Kaggle's free GPU limits. Justify your cleaning decisions in the report.

Task 3: Select & Load a Model from Hugging Face [10 marks]

Choose a small, task-appropriate pre-trained model from Hugging Face Hub.

- Load the tokenizer for your chosen model [3 marks]
- Load the model with the correct number of output labels from your id2label mapping [3 marks]
- Write a short paragraph (100–150 words) in the report explaining why you chose this model — reference the Hugging Face model card [4 marks]

💡 **Tip:** Pick a model under 200 MB so it trains comfortably within Kaggle's free GPU hours. Models such as DistilBERT, DistilRoBERTa, or similar compact variants are good starting points. Avoid large models (GPT-2 large, LLaMA, etc.) — they will exhaust free GPU quota quickly. Read the model card before writing your justification paragraph.

Task 4: Train Multiple Versions on Kaggle & Track with W&B [25 marks]

Fine-tune your model inside a Kaggle Notebook using the Hugging Face Trainer API. Run at least two experiment versions that differ in at least one hyperparameter.

- Import your notebook into Kaggle and enable GPU (Settings → Accelerator → GPU T4 x2) [3 marks]
- Store WANDB_API_KEY and HF_TOKEN using Kaggle Secrets — do not hardcode tokens [3 marks]
- Run at least two training versions with different hyperparameter configurations [7 marks]
- Log training loss, validation loss, accuracy, F1, and all hyperparameters to W&B for every run [7 marks]
- Include a screenshot of your W&B dashboard showing both runs in the report [5 marks]

Step 1: Load Secrets in Kaggle

```
from kaggle_secrets import UserSecretsClient
import os, wandb
from huggingface_hub import login
```

```
secrets = UserSecretsClient()
os.environ['WANDB_API_KEY'] = secrets.get_secret('WANDB_API_KEY')
login(token=secrets.get_secret('HF_TOKEN'))
wandb.login()
```

Step 2: Initialise a W&B Run

```
import wandb

wandb.init(
    project='mlops-assignment3',
    name='run-v1',          # change for each version
    config={
        'model':      model_name,
        'epochs':      3,
        'batch_size':  16,
        'learning_rate': 3e-5,
        'version':      'v1',
        'platform':     'Kaggle',
    }
)
```

Step 3: Training Arguments (include report_to='wandb')

```
from transformers import TrainingArguments, Trainer
from sklearn.metrics import accuracy_score, f1_score

def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    return {
        'accuracy': accuracy_score(labels, preds),
        'f1':      f1_score(labels, preds, average='weighted'),
    }

training_args = TrainingArguments(
    output_dir      = './results',
    num_train_epochs= 3,
    per_device_train_batch_size = 16,
    eval_strategy   = 'epoch',
    save_strategy   = 'epoch',
    load_best_model_at_end = True,
    report_to       = 'wandb',
    run_name        = 'run-v1',
)

trainer = Trainer(
    model=model, args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
trainer.train()
wandb.finish()
```

💡 **Tip:** Kaggle gives 30 hours/week of free GPU. Change one hyperparameter at a time between versions so your W&B comparison is meaningful.

Task 5: Push the Trained Model to Hugging Face Hub [5 marks]

Push your best trained model to your public Hugging Face profile so GitHub Actions can load it for inference.

- Push the model weights and tokenizer to a public HF repository [3 marks]
- Log the Hugging Face model URL into your W&B run summary [2 marks]

```
from huggingface_hub import login
login(token=HF_TOKEN) # loaded from Kaggle Secrets

model.push_to_hub('your-username/your-model-repo')
tokenizer.push_to_hub('your-username/your-model-repo')

wandb.run.summary['huggingface_model'] = \
    'https://huggingface.co/your-username/your-model-repo'
```

🔴 **Note:** Set the repository visibility to Public on Hugging Face before submitting — the GitHub Actions inference workflow pulls the model from this URL.

Task 6: Create a Dockerfile [10 marks]

Write a Dockerfile that packages your inference code so anyone can run predictions without manual setup.

- Use a slim Python base image and install only inference dependencies [3 marks]
- Accept the HF model name as a build argument (ARG) with a sensible default [2 marks]
- Build and test the image locally — confirm it runs end-to-end [3 marks]
- Push the image to a public registry (Docker Hub or GHCR) and paste the URL in your report [2 marks]

```
# Build
docker build --build-arg HF_MODEL_NAME=your-username/your-model \
    -t mlops-a3-inference:latest .

# Test
docker run --rm -e HF_TOKEN=<token> -e INPUT_TEXT='Sample text' \
    mlops-a3-inference:latest
```

```
# Push
docker push your-dockerhub/mlops-a3-inference:latest
```

Task 7: Push to GitHub & Trigger GitHub Actions [15 marks]

All scripts, the Dockerfile, and the workflow files must be in the repository. Two workflows are required.

⚠ Important: GitHub Actions is used only for CI (linting) and inference — NOT for training. All training must be done on Kaggle Notebooks. Do not attempt to run training inside a GitHub Actions workflow.

7.1 CI Workflow — triggers on push to develop [5 marks]

```
# .github/workflows/ci.yml
name: CI
on:
  push:      { branches: [develop] }
  pull_request: { branches: [main] }
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt flake8
      - run: flake8 src/ --max-line-length=120
```

7.2 Inference Workflow — triggers manually [7 marks]

```
# .github/workflows/inference.yml
name: Inference
on:
  workflow_dispatch:
    inputs:
      input_text:
        description: 'Text to classify'
        required: true
jobs:
  infer:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt
      - env:
          HF_TOKEN: ${ secrets.HF_TOKEN }
          INPUT_TEXT: ${ github.event.inputs.input_text }
```

```
run: python src/inference.py
```

7.3 GitHub Secrets [3 marks]

- Add HF_TOKEN and WANDB_API_KEY via Settings → Secrets and Variables → Actions
- Never commit API tokens to the repository

Task 8: Show All Experiments on W&B [5 marks]

- Both training runs must appear in the W&B project dashboard
- Set the project visibility to Public in W&B Settings
- Use the Runs Comparison table to show Accuracy, F1, and Loss side-by-side
- Paste the W&B project URL in your report

README Checklist

- Setup instructions (how to install and run each script)
- Links to both public Kaggle notebooks
- Link to Hugging Face model
- Link to Docker image
- Link to W&B project dashboard
-

Mark Summary

Task	Marks
Task 1: Set Up the GitHub Repository	10
Task 2: Data Cleaning & Normalisation	15
Task 3: Select & Load a Model from Hugging Face	10
Task 4: Train Multiple Versions on Kaggle & Track with W&B	25
Task 5: Push Trained Model to Hugging Face Hub	5
Task 6: Create a Dockerfile	10
Task 7: Push to GitHub & Trigger GitHub Actions	15
Task 8: Show All Experiments on W&B	5
Report (PDF)	5
TOTAL	100

A Note to Students

The goal of this assignment is not to achieve perfect accuracy — it is to demonstrate command of the full MLOps workflow. Your marks come from how well you build, containerise, automate, and document the pipeline, not from the model's final score.

- Treat the model as a black box with inputs and outputs — your job is the pipeline around it
- Justify every decision you make: cleaning choices, model selection, hyperparameter changes
- Every link in your report must be live and publicly accessible at submission time

 **GPU Note:** Use Kaggle Notebooks (free tier) for training — enable GPU T4 under Accelerator. If GPU hours run low, reduce dataset sample size or the number of training epochs.